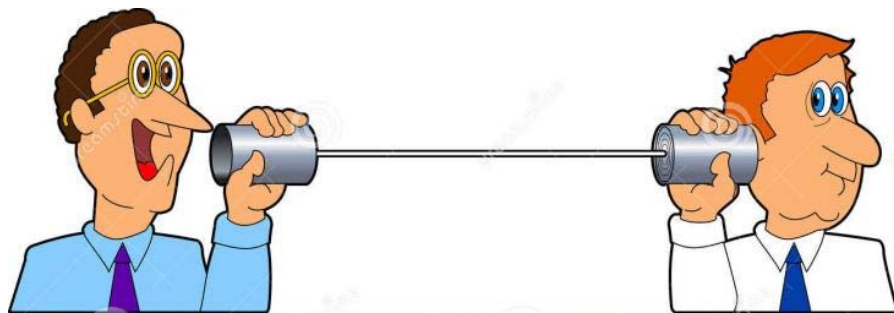


《单片机原理及应用》 第九讲： UART通信接口





9.1 串行通信及基础知识



C8051F具有丰富的串行通信接口。包括**2个UART**通信接口、一个与I²C兼容的**SMBus**接口和一个**SPI**接口。

UART: (Universal Asynchronous Receiver/Transmitter)

通用异步收发传输器

- UART串行接口是全双工串行通信接口，即能**同时进行串行发送和接收**。它可以作UART（通用异步接收和发送器）用，也可以作同步位移寄存器用。
- 应用UART串行接口可以实现C8051F单片机系统之间点对点的**单机通信、多机通信**以及C8051F与系统机（如IBM-PC机等）的**单机或多机通信**。



1. 数据通信的概念

• 并行通信和串行通信

- **并行通信**是指数据的**各位同时**进行传送（发送或接收）的通信方式。其优点是传递速度快；缺点是数据有多少位，就需要多少根传送线。并行通信在位数多、传送距离又远时就不太适宜。

- **串行通信**指数据是**一位一位按顺序**传送的通信方式，它的突出优点是只需一对传送线（利用电话线就可作为传送线），这样就大大降低了传送成本，特别适用于远距离通信。其缺点是传送速度较低。



2. 串行通信的传送方向

• 单工 (Simplex)

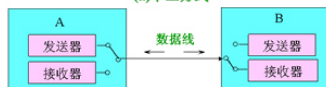
- 只允许数据向一个方向进行传送;



(a)单工方式

• 半双工 (Half-duplex)

- 允许数据向两个方向中的任何一个方向传送，但一次只能有一个发送，一个接收;



(b)半双工方式



(c)全双工方式

• 全双工 (Full-duplex)

- 允许同时双向传送数据，因此，全双工配置是一对单工配置，它要求两端的通信设备都具有完整和独立的发送和接收能力。



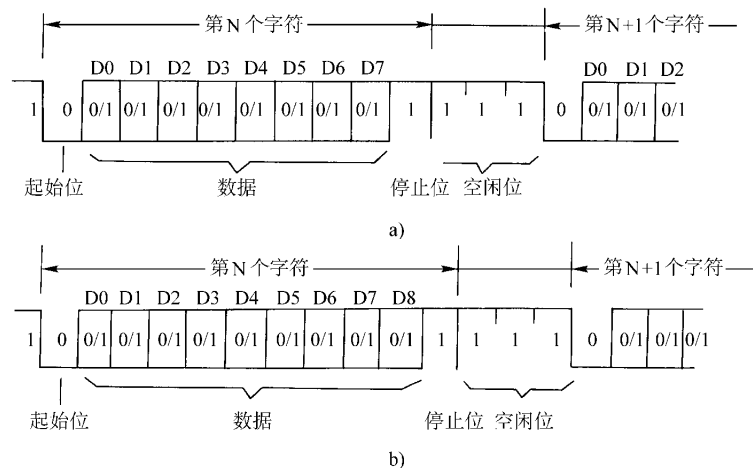
3. 异步通信和同步通信

(1) 异步通信

- 异步通信用**起始位0**表示字符的开始，然后**从低位到高位**逐位传送数据，最后用**停止位1**表示字符结束。一个字符又称一**帧**信息。一帧信息包括1位起始位、 n 位数据位和1位停止位。
- 帧与帧之间可以无间隔，也可以有间隔，间隔时间可以任意改变，间隔位用空闲位1来填充。



异步通信



在C8051F单片机系统中，第9位数据D8可以用作奇偶校验位，也可以用作地址/数据帧的标识位，D8=1表示该帧信息传送的是地址，D8=0表示传送的是数据。



• 异步通信必须事先约定：

➤ 字符格式：字符的编码形式（如ASCII码等）、奇偶校验、波特率

➤ 波特率
baud/s,
双方都以

BASIS FOR COMPARISON	BIT RATE	BAUD RATE
Basic	Bit rate is the count of bits per second.	Baud rate is the count of signal units per second.
Meaning	It determines the number of bits traveled per second.	It determines how many times the state of a signal is changing.
Term usually used	While the emphasis is on computer efficiency.	While data transmission over the channel is more concerned.
Bandwidth determination	Can not determine the bandwidth.	It can determine how much bandwidth is required to send the signal.
Equation	Bit rate = baud rate x the count of bits per signal unit	Baud rate = bit rate / the number of bits per signal unit

约定

, 单位是
通信要求收发

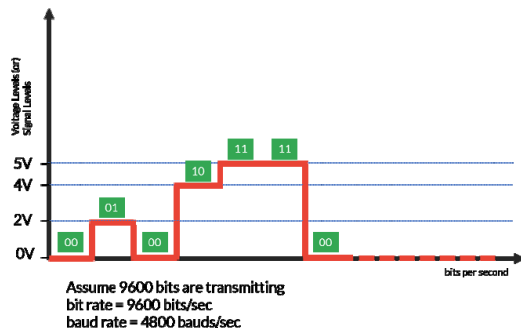
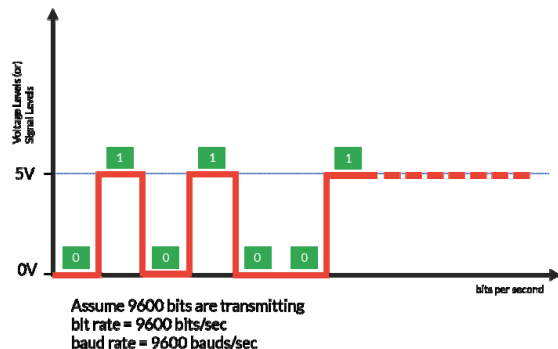
• 异步通信必须事先约定：

- 字符格式：字符的编码形式（如ASCII码等）、奇偶校验形式，以及起始位和停止位的规定
- 波特率（Baud rate）：数据传送速率，单位是 baud/s，每秒传送的信号变化。异步通信要求收发双方都以同样的波特率工作。



波特率 VS 比特率

➤波特率 VS 比特率：对于2值（0和1）的**比特信号**，由于信号的变化数量等于传输的比特数，因此**波特率与比特率相同**。但**多值**信号时不等同。



假设数据串行传输的速率是120字符每秒，一个字符由10位二进制信息组成：1位起始位+8位数据位+1位停止位。其波特率为：

$$10\text{bit/字符} \times 120\text{字符/s} = 1200\text{bits/s} \quad (1200\text{bauds/s})$$



(2) 同步通信

- 在同步通信中，每一数据块开头时发送一个或两个**同步字符**，使发送与接收双方取得同步。数据块的各个字符间取消了起始位和停止位，所以通信速度得以提高，如图所示。同步通信时，如果发送的数据块之间有间隔时间，则发送同步字符填充。

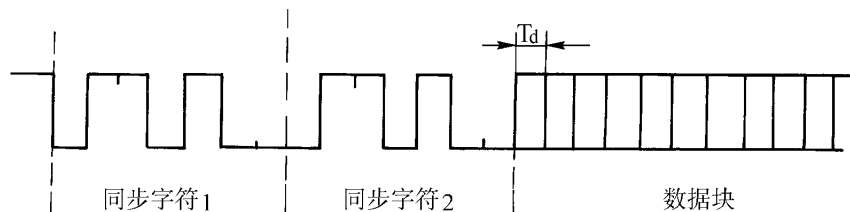


图4-19 同步通信数据格式



9.2 串行接口的组成和特性

C8051F020有两个**增强型**全双工UART串行通信接口：

➤ UART0和UART1

➤ 都具有帧错误检测和多机通信地址硬件识别功能

➤ 可以工作在全双工异步方式或半双工同步方式

➤ 支持多处理器通信。

➤ **UART0以T1或T2作为波特率发生器，UART1以T1或T4作为波特率发生器。**



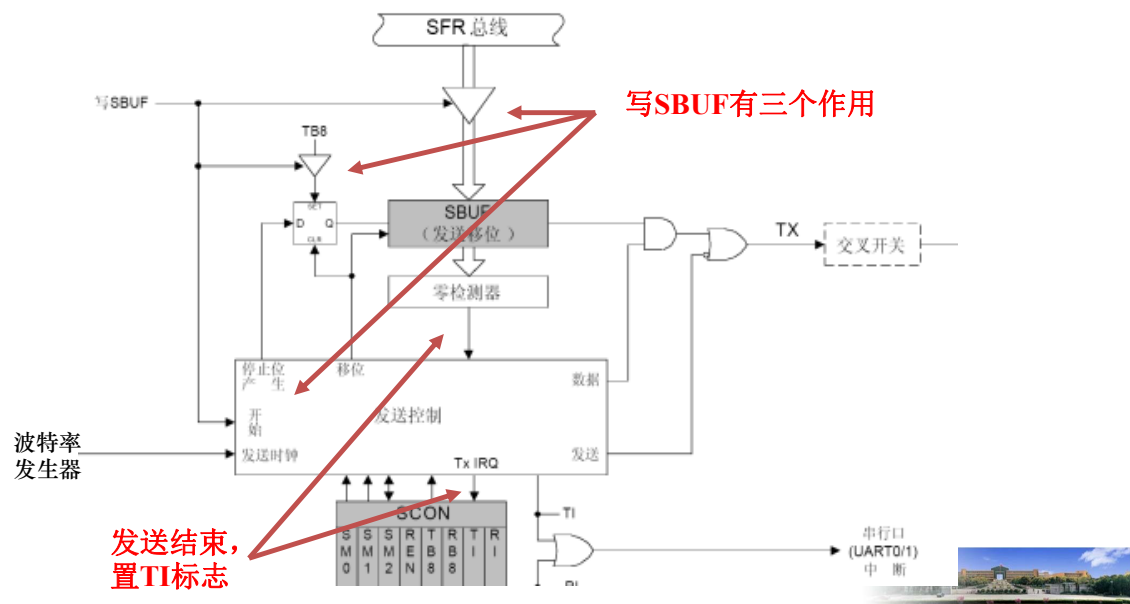
1. UART的组成

- 串行口的内部有**独立**的数据接收缓冲器和数据发送缓冲器。
- 数据接收缓冲器**只能读出不能写入**，数据发送缓冲器**只能写入不能读出**
- 这两个数据缓冲器都用符号SBUF_n来表示（SBUF0和SBUF1表示）的， SBUF0地址都**（物理上是两个片区）**是99H（SBUF1的地址都是F2H）。
- CPU对特殊功能寄存器SBUF0或SBUF1执行**写操作**，就是将数据写入发送缓冲器；对SBUF0或SBUF1的**读操作**，就是读出接收缓冲器的内容。



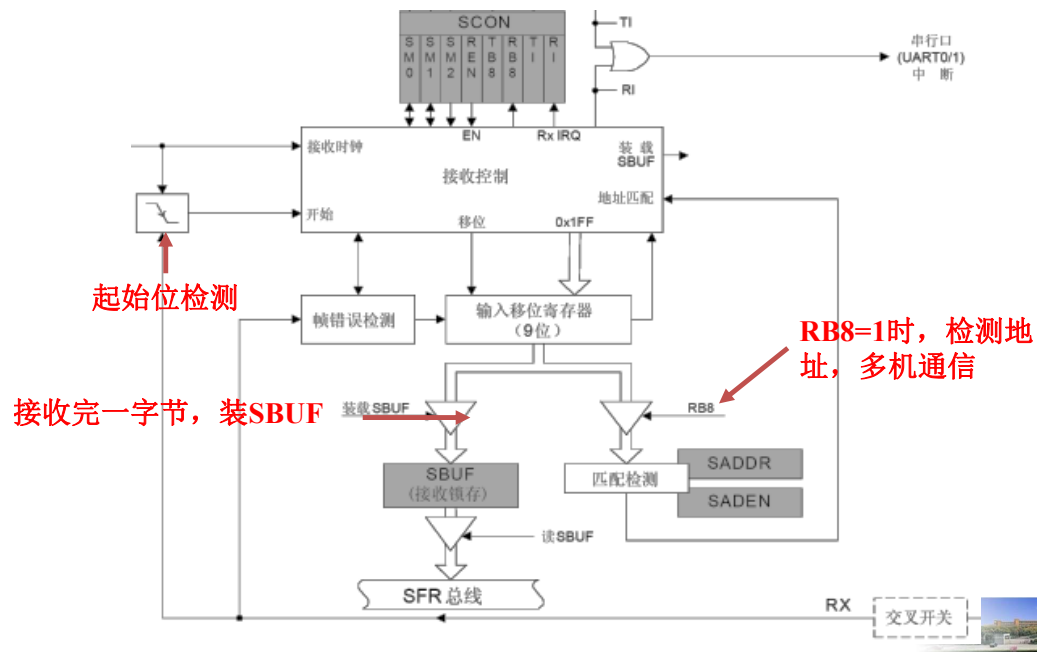
UART的原理框图

发送电路由发送SBUF、零检测器和发送控制器等组成。



UART的组成—接收部分

接收电路由接收SBUF、接收移位寄存器和接收控制器等组成。



2、UART的引脚配置

RX0和TX0配置的端口不受P0MDOUT影响，总是漏极开路



3. 串行口控制寄存器SCON

D7	D6	D5	D4	D3	D2	D1	D0	
SM00/FE0	SM10/RX OV0	SM20/TXC OL0	REN0	TB80	RB80	TI0	RI0	字节 地址 98H

•位7-5: 该三位的功能由PCON中的SSTAT0位决定

-SSTAT0=1时, 用于UART0的出错状态指示位。

- FE0=1为帧格式错, 即收到的停止位为0;
- RXOV0=1为接收覆盖错误, 即前一个字节还没有取走, 接收器又锁存了一个新字节;
- TXCOL0=1为发送冲突错误, 即前一个字节还没有发送完又往SBUF写入一新字节。

-SSTAT0=0时, SM00、SM10按下页表选择UART0的工作方式。默认值为0, 同



SM00	SM10	方式	功 能 说 明
0	0	0	同步方式（扩展移位寄存器方式，用于I/O口扩展），波特率固定（ $f_{osc} / 12$ ）
0	1	1	8位UART，波特率可变（由T1或T2溢出率决定）
1	0	2	9位UART，波特率固定（ $f_{osc} / 64$ 或 $f_{osc} / 32$ ）
1	1	3	9位UART，波特率可变（由T1或T2溢出率决定）



D7	D6	D5	D4	D3	D2	D1	D0	字节 地址 98H
SM00/FE0	SM10/RX OV0	SM20/TXC OL0	REN0	TB80	RB80	TI0	RI0	

•SM20为多处理器通信允许位

—方式0时该位无作用。

—方式1时用于检查有效停止位。

•0：忽略对停止位的检测。

•1：只有接收到有效的停止位时才置位RI0。

—方式2和方式3时为多机通信允许位

•0：忽略对第九位的检测。

•1：只有当第九位为“1”并且接收到的地址与UART0的地址或广播地址匹配时才置位RI0并产生中断。



D7	D6	D5	D4	D3	D2	D1	D0	字节 地址 98H
SM00/FE0	SM10/RX OV0	SM20/TXC OL0	REN0	TB80	RB80	TI0	RI0	

•位4 (**REN0**) : 接收允许位

—0: 禁止UART0接收 1: 允许UART0接收

•位3 (**TB80**) : 第9发送位

—对于方式2和方式3, 是发送的第9位数据。可跟据需要时由软件置位或复位。在方式0和1中未用。

•位2 (**RB80**) : 第9接收位

—对于方式2和方式3, 是接收到的第9位数据。对于方式1, 如SM20=0, RB80是接收到的停止位。对于方式0, 不使用RB80。



D7	D6	D5	D4	D3	D2	D1	D0	字节 地址 98H
SM00/FE0	SM10/RX OV0	SM20/TXC OL0	REN0	TB80	RB80	TI0	RI0	

• 位1 (TI0) : 发送中断标志位

—由硬件在方式0串行发送第8位结束时置位，或在其它方式串行发送停止位的开始时置位。当允许UART0中断时，置1该位将使CPU转到UART0中断服务程序。该位必须由软件清0。

• 位0 (RI0) : 接收中断标志位

—由硬件在方式0接收到第8位结束时置位，或在其它方式接收到停止位的中间时置位。当允许UART0中断时，置1该位将使CPU转到UART0中断服务程序。该位必须由软件清0。



4、电源控制寄存器PCON

D7	D6	D5	D4	D3	D2	D1	D0	字节地址 87H
SMOD0	SSTAT0	-	SMOD1	SSTAT1	-	STOP	IDLE	

•位7 (**SMOD0**) : UART0波特率系数控制位

-0: UART0的波特率不加倍。

-1: UART0的波特率加倍。

•位6 (**SSTAT0**) : UART0增强状态方式选择位

-0: 读/写SCON0.5~7时访问UART0方式设置位SM20-SM00。

-1: 读/写SCON0.5~7时访问帧格式错误 (FE0)、接收覆盖错误 (RXOV0) 和发送冲突错误 (TXCOL0) 状态位。

•位5: 保留。读出值无定义, 写入必须为0。



D7	D6	D5	D4	D3	D2	D1	D0	字节地址 87H
SMOD0	SSTAT0	-	SMOD1	SSTAT1	-	STOP	IDLE	

•位4 (**SMOD1**) : UART1波特率系数控制位

-0: UART1的波特率不加倍。

-1: UART1的波特率加倍。

•位3 (**SSTAT1**) : UART1增强状态方式选择。

-0: 读/写SCON1.5~7时访问UART1方式设置位SM21-SM01。

-1: 读/写SCON1.5~7时访问帧格式错误 (FE1)、接收覆盖错误 (RXOV1) 和发送冲突错误 (TXCOL1) 状态位。

•位2: 保留。读出值无定义, 必须写入0。

•位1-位0: 见2.6节电源管理方式。



9.3 串行接口的工作方式

1. 方式0

• 方式0是半双工方式，扩展移位寄存器工作方式，用以扩展并行I/O接口。

— 串行发送，发送SBUF相当于一个并入串出移位寄存器，由C8051F的内部总线并行接收8位数据，并从RXD线串行输出。

— 串行接收，接收SBUF相当于一个串入并出的移位寄存器，从RXD线接收一帧串行数据，并把它并行地送到内部总线。

• 由RXD串行输入/输出数据，TXD输出移位脉冲。波特率固定为 $f_{osc}/12$ 。

• 数据帧固定为8位，低位在前高位在后，没有起始位和停止位

• RXn被强制为漏极开路方式，通常需要外接一个上拉电阻

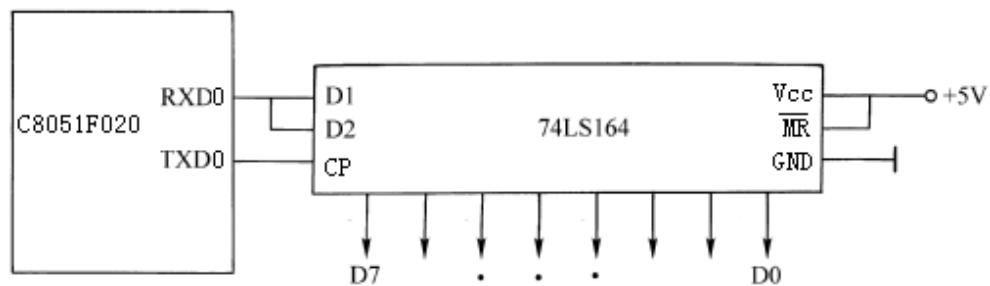


(1) 方式0发送

- 串口外接74LS164串入并出移位寄存器的接口逻辑如下页图所示。TXD输出移位脉冲，RXD输出数据移入74LS164。
- CPU写发送SBUF，启动串行口从低位开始发送，经过8个发送周期，发送SBUF的内容移入74LS164，并置位TI，完成一字节输出。
- 若 $f_{osc}=12\text{MHz}$ ，则串行输出一位的时间是 $1\mu\text{s}$ ，传输一个字节需 $8\mu\text{s}$ 。
- 从低位开始串行输出，数据的低位在右高位在左，在具体应用中应加以注意。方式0输出时，可以串接多个移位寄存器。



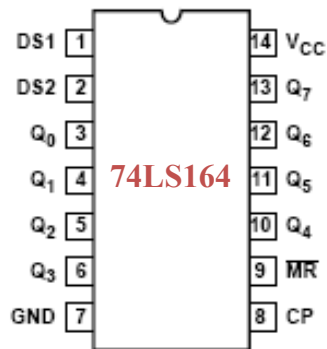
(1) 方式0发送



(1) 方式0发送

PIN NAMES

A, B	Data Inputs
CP	Clock (Active HIGH Going Edge) Input
$\overline{MR}$	Master Reset (Active LOW) Input
$Q_0 - Q_7$	Outputs



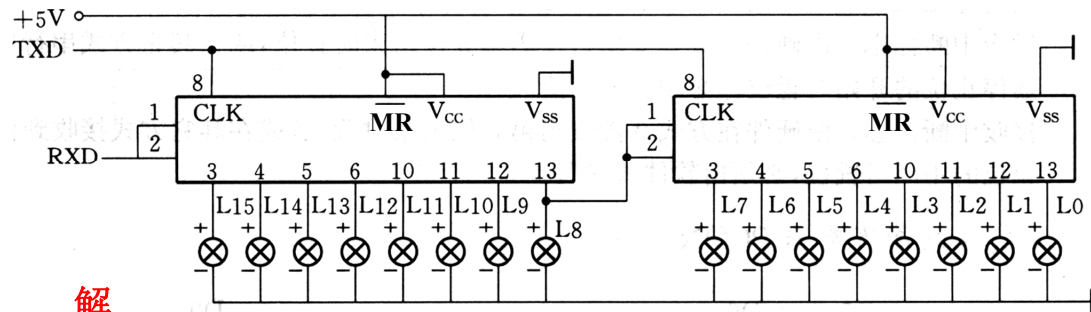
74LS164真值表

OPERATING MODE	INPUTS				OUTPUTS	
	$\overline{MR}$	CP	DS1	DS2	Q_0	$Q_1 - Q_7$
RESET (CLEAR)	L	X	X	X	L	L - L
Shift	H	$\uparrow$	l	l	L	$q_0 - q_6$
	H	$\uparrow$	l	h	L	$q_0 - q_6$
	H	$\uparrow$	h	l	L	$q_0 - q_6$
	H	$\uparrow$	h	h	H	$q_0 - q_6$



(1) 方式0发送举例

例1 编程使下图中L0~L3、L8、L10、L12、L14亮，其余灭



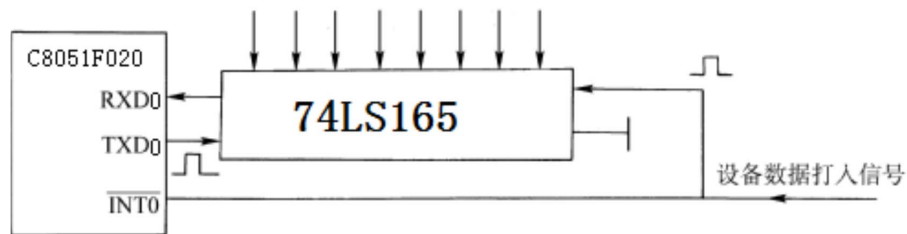
解

```
SCON0=0x00;    //UART0初始化, 方式0, 禁止接收
SBUF0=0x0f;     //发送一字节
while (!TI0);   //等待发送完毕
TI0=0;          //清除发送结束标志
SBUF0=0x55;     //继续发下一字节
while (!TI0);
TI0=0;
.....;        //其他操作
```



(2) 方式0接收

- 方式0输入，RXD--数据输入线，TXD--移位脉冲，串行口与外接的并入串出的移位寄存器74LS165的接口逻辑如图所示。



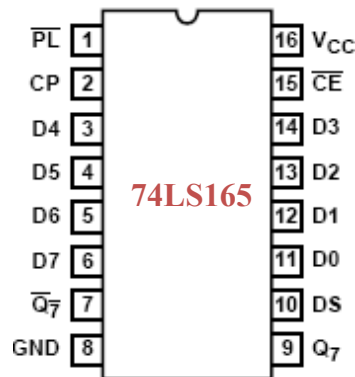
方式0输入时移位寄存器的连接



(2) 方式0接收举例

• 74LS165并入串出移位寄存器

/PL	移位/置位控制，低电平置位，高电平移位
CP	时钟输入端，
D0~D7	并行输入端
Q7	串行输出端，/Q7：串行输出的反相
DS	串行输入端，用于扩展多个74LS165
/CE	时钟禁止端，低电平时允许时钟输入

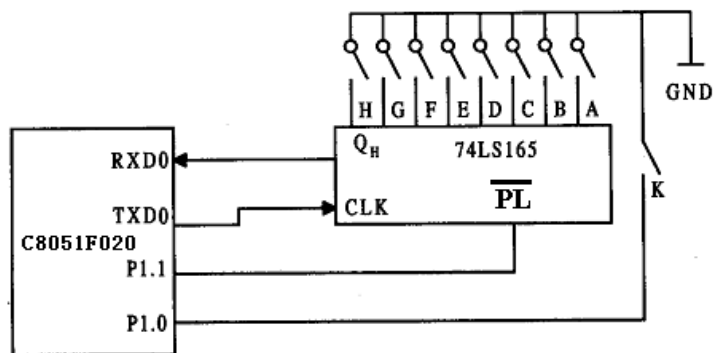


74LS165真值表

OPERATING MODE	INPUTS					Q _n REGISTER		OUTPUTS	
	PL	CE	CP	DS	D0 - D7	Q ₀	Q ₁ - Q ₆	Q ₇	$\overline{Q}_7$
Parallel Load	L	X	X	X	L	L	L-L	L	H
	L	X	X	X	H	H	H-H	H	L
Serial Shift	H	L	↑	l	X	L	q ₀ - q ₆	q ₆	$\overline{q}_6$
	H	L	↑	h	X	H	q ₀ - q ₆	q ₆	$\overline{q}_6$
Hold Do Nothing	H	H	X	X	X	q ₀	q ₁ - q ₆	q ₇	$\overline{q}_7$

(2) 方式0接收举例

例2：根据下图编写程序，当开关K合上时，采集8位开关量，根据开关的不同状态进行不同处理。



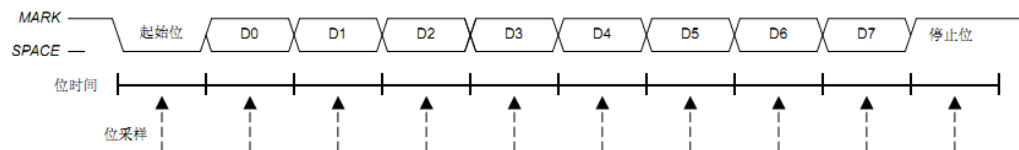
(2) 方式0接收举例

```
while(P1^0);      //等待开关合上
P1^1=0;           //74LS165并行输入数据
P1.1=1;           //开始串行移位
SCON=0x10;        //串行口方式0，并启动接收
while(!RI);       //等待接收完成
RI=0;             //清除接收完成标志
A=SBUF;           //将接收到的数据给变量A
    ⋮             //根据变量A的数值进行不同操作
```



2. 方式1

• 方式1是8位异步串行通信方式，TXD为数据输出线，RXD为数据输入线。传送一帧信息的数据格式如下图所示，一帧为10位：1位起始位，8位数据位（先低位后高位）和1位停止位。



2、方式1

(1) 方式1发送

CPU向发送SBUF写入一个数据，即启动发送，从TXD端输出一帧信息，先发送起始位0，接着从低位开始依次输出8位数据，最后输出停止位1，并置1发送中断标志TI。

CPU查询TI=1后，清TI，再向SBUF写入数据，启动下一字符发送。也可以采用中断方式，TI=1时向CPU产生中断请求。



(2) 方式1接收

• **REN置1**允许接收器接收。

• 接收机制（了解即可）：（1）接收器以所选波特率的16倍的速率采样RXD端的电平。当检测到RXD端输入电平发生**负跳变**时，复位内部的十六分频计数器。

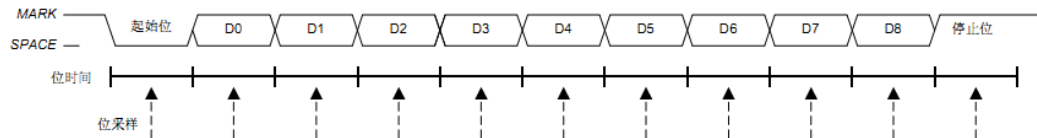
（2）计数器的16个状态把传送一位数据的时间分为16等分，在每位中心，即7、8、9这三个计数状态，位检测器采样RXD的输入电平，接收的值是**三次采样中至少是两次相同**的值，这样处理可以防止干扰。

（3）如果第1位时间接收到的值（起始位）不是0，则起始位无效，复位接收电路，重新搜索RXD端上的负跳变。接收到停止位为1时，将接收到的8位数据装入接收数据缓冲器SBUF，置位RI，供CPU查询或向CPU请求中断。



3. 方式2和方式3

- 方式2和方式3是**9位异步串行通信**方式，TXD为数据发送端，RXD为数据接收端。方式2的波特率固定为振荡器频率的 $1/64$ 或 $1/32$ ，而方式3的波特率由定时器T1或T2的溢出率确定（UART1方式3的波特率由定时器T1或T4的溢出率确定）。
- 在方式2和方式3中，一帧信息为11位：1位起始位，8位数据位（先低位后高位），1位附加的第9位数据（发送时为SCON中的TB8，接收时为SCON中的RB8。用于**奇偶校验和多机通信**），1位停止位。数据的格式如下图所示。



3、方式2和方式3

- 多一附加位TB8、RB8，可用于多机通信和奇偶校验
- **区别**是波特率不同
 - 方式2的波特率固定为振荡器频率的1/64或1/32,
 - 方式3的波特率由定时器T1或T2 (UART0) / T1或者T4 (UART1) 的溢出率确定
- 发送：写发送SBUF即启动发送→发完置位TI
- 接收：置位REN →接收完后，若**RIn=0**，**SM2n=0**或**收到**的**第9位为1**，则置位RIn →读接收SBUF



9.4 波特率设计

- 方式0: $f_{OSC} / 12$
- 方式2: $2^{SMOD} \times f_{OSC} / 64$
- 方式1、3: $2^{SMOD} \times T1\text{溢出率} / 32$ (若用定时器T1产生)

T1 时钟选择位
(CKCON.4)

$$\therefore T1\text{溢出周期} = \frac{256 - X}{SYSCLK \times 12^{(T1M-1)}}$$

方式2 方式2的溢出周期时间

$$\therefore T1\text{溢出率} = \frac{SYSCLK \times 12^{(T1M-1)}}{256 - X}$$

已知波特率,求定时器初值X

$$\therefore \text{波特率} = \left(\frac{2^{SMOD0}}{32} \right) \times \left(\frac{SYSCLK \times 12^{(T1M-1)}}{256 - X} \right)$$

★

$$\therefore X = 256 - \frac{2^{SMOD0} \times SYSCLK \times (12^{(T1M-1)})}{32 \times \text{波特率}}$$



•**例3** 已知C8051F单片机时钟振荡频率为11.0592MHz，选用定时器T1工作方式2作波特率发生器，波特率为2400 bauds/s，求初值X。

•**解：**设波特率控制位SMOD0=0，定时器T1计数脉冲控制位T1M=0，则有：

$$X=256 - \frac{11.0592 \times 10^6 \times (12^{-1})}{32 \times 2400} = 244 = F4H$$

P153 常用波特率表



(2) 用定时器T2和T4产生波特率

• C/T2 = 0 时，波特率计算公式为：

$$\text{方式 1 和方式 3 波特率} = \frac{SYSCLK}{32 \times (65536 - [RCAP2H: RCAP2L])}$$

• C/T2 = 1 时，波特率计算公式为：

$$\text{方式 1 和方式 3 波特率} = \frac{F_{CLK}}{16 \times (65536 - [RCAP2H: RCAP2L])}$$

F_{CLK} 为 T2 引脚上输入信号的频率

T4 作波特率发生器类似



9.5 串行口应用编程

- **交叉开关配置。**将TXn和RXn配置到相应的I/O端口
- **设定波特率。**串行口的波特率有两种，即固定波特率和可变波特率。当使用可变波特率时，应先计算T1或T2/T4的计数初值，并对相应定时器进行初始化；如使用固定波特率（方式0、方式2），则此步骤可以省略。
- **填写控制字。**即对SCON寄存器设定工作方式，如果是接收程序或双工通信方式，需要置REN=1（允许接收），同时也要将TI、RI进行清零。



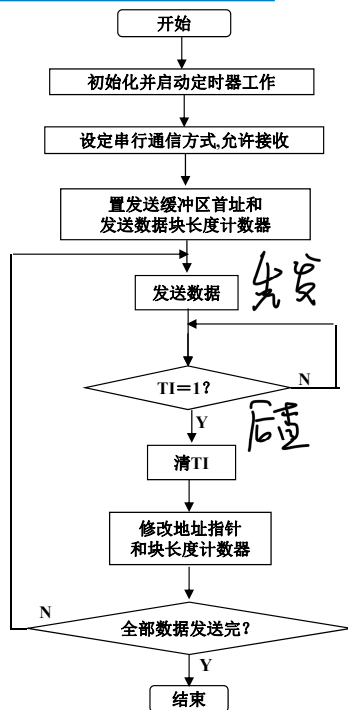
➤ 采用**查询和中断**两种方式。TI和RI是**一帧数据是否发送完或收齐的标志**，可用于CPU查询；如果允许，也可以向CPU申请中断。**两种方式的编程方法如下：**

- **查询式发送程序**：发送一个数据→查询TI→发送下一个数据（先发后查）。
- **查询式接收程序**：查询RI→读入一个数据→查询RI→读下一个数据（先查后收）。
- **中断式发送程序**：发送一个数据→等待中断→在中断服务程序中发送下一个数据。
- **中断式接收程序**：等待中断→在中断服务程序中接收一个数据。

➤ 两种方式中，发送和接收数据后都需要注意**清零TI或RI**

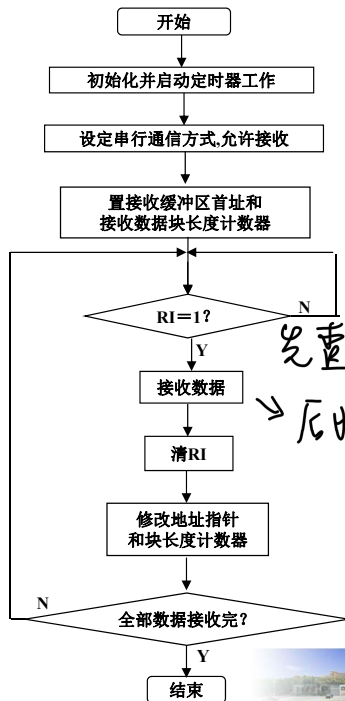


1. 查询方式

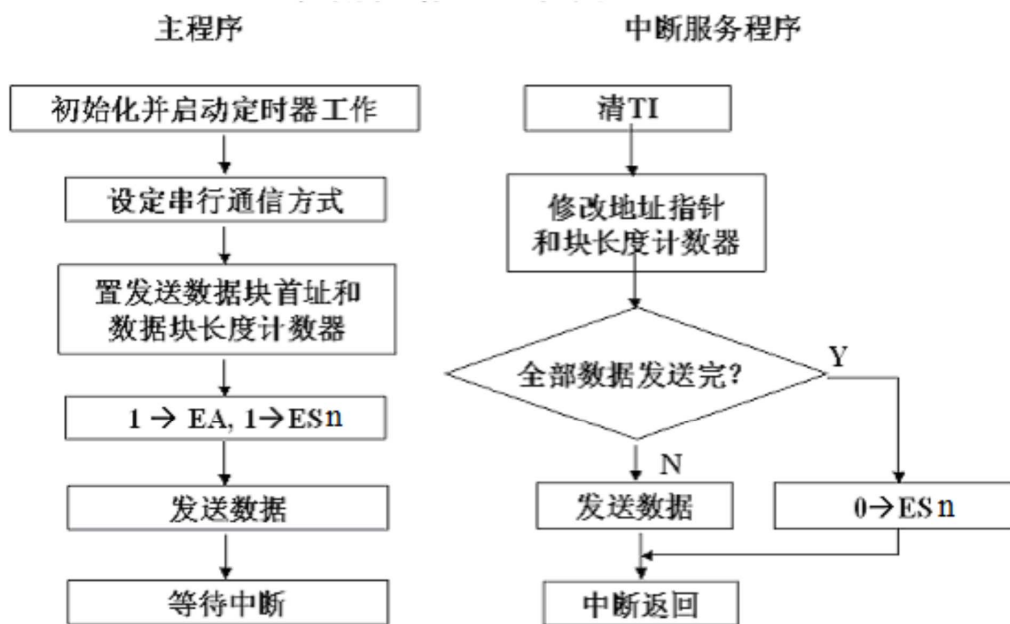


初始化

循环发送或接收

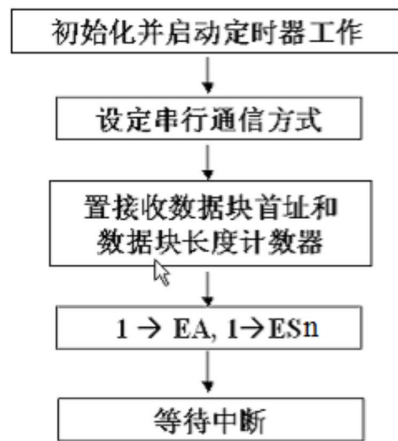


2. 中断方式发送

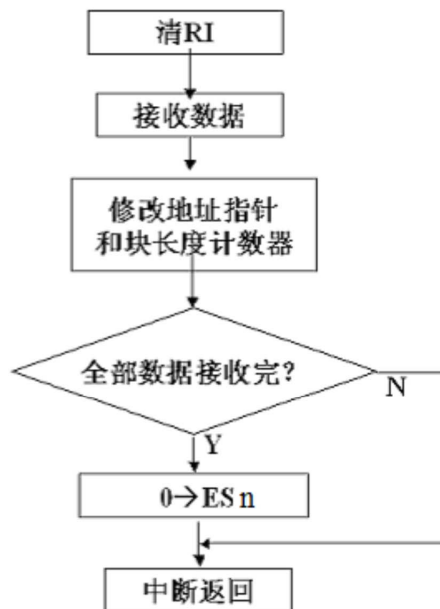


2. 中断方式接收

主程序



中断服务程序



3. 串行通信编程举例

• **例4** 在C8051F020的片内数据存储器20H~3FH单元共有32个字节数据，要求使用UART0方式1发送出去，传输波特率为9600，设SYSCLK=12MHz。试分别用查询方式和中断方式编写发送程序和相应的接收程序。

• **解** 选T1工作于方式2，作为波特率发生器，取T1M=0（T1按振荡器的12分频计数）、SMOD0=0，T1的时间常数计算

253 (FDH) (四舍五入)

$$\therefore X = 256 - \frac{2^{SMOD0} \times SYSCLK \times (12^{(T1M-1)})}{32 \times \text{波特率}}$$



查询方式发送程序：

```
#include <c8051f020.h>
void main(void)
```

```
{
    unsigned char i;
    char data *p;
    XBR0=0x04;
    XBR2=0x40;
    P0MOUT|=0x01;
    TMOD=0x20;
    TH1=0xFD;
    TL1=0xFD;
    TR1=1;
    SCON0=0x40;
    p=0x20;
    for(i=0;i<32;i++)
    {
        SBUF0=*p;
        p++;
        while(!TI0);
        TI0=0;
    }
}
```

写也设用 →

P0MOUT|=0x01;

//发送数据块地址指针

//配置交叉开关，使能UART0

//使能交叉开关

//TX0为推挽输出（错），总是漏极开路

//初始化并启动T1，方式2

→ T1方式2 自动重装初值，有效只有8位。

//UART0初始化，方式1

//地址指针初始化

先发后查

//一字节送发送SBUF0

//等待发送完成

题目要求数 →
放在第一个
地址



查询方式接收程序:

```
#include <c8051f020.h>
void main(void)
{
    unsigned char i;
    char data *p;           //发送数据块地址指针
    XBR0=0x04;
    XBR2=0x40;
    P0MOUT|=0x01; //TX0为推挽输出(错), 总是漏极开路
    TMOD=0x20;           //初始化并启动T1
    TH1=0xfd;
    TL1=0xfd;
    TR1=1;
    SCON0=0x50;           //UART0初始化, 方式1、允许接收
    p=0x20;               //地址指针初始化
    for(i=0;i<32;i++)
    {
        while(!RI0);       //等待UART0接收一个字符
        RI=0;
        *p=SBUF0;           //放入接收缓冲区
        p++;
    }
}
```



中断方式发送主程序：

```
#include <c8051f020.h>
char data *p;           //发送数据块地址指针
void main(void)
{
    XBR0=0x04;           //配置交叉开关
    XBR2=0x40;
    P0MOUT|=0x01; //TX0为推挽输出（错），总是漏极开路
    TMOD=0x20;           //初始化并启动T1
    TH1=0xfd;
    TL1=0xfd;
    TR1=1;
    SCON0=0x40;          //UART0初始化，方式1
    p=0x20;              //地址指针初始化
    EA=1;                //开中断
    ES0=1;
    SBUF0=*p;            //发送第一个字符
    while(1);           //等待发送中断
}
```



中断方式发送中断服务程序:

```
void interrupt_UART0(void) interrupt 4
{
    TI0=0;           //清发送中断标志
    p++;
    if(p<0x40)
        SBUF0=*p;    //发送下一字节
    else
        ES0=0;       //关串口中断
}
```



中断方式接收主程序：

```
#include <c8051f020.h>
char data *p;           //发送数据块地址指针
void main(void)
{
    unsigned char i;
    XBR0=0x04;           //配置交叉开关
    XBR2=0x40;
    P0MOUT|=0x01; //TX0为推挽输出（错），总是漏极开路
    TMOD=0x20;           //初始化并启动T1
    TH1=0xfd;
    TL1=0xfd;
    TR1=1;
    SCON0=0x50;           //UART0初始化，方式1、允许接收
    p=0x20;               //地址指针初始化
    EA=1;                 //开串口中断
    ES0=1;
    while(1);             //等待接收中断
}
```



中断方式接收中断服务程序:

```
void interrupt_UART0(void) interrupt 4
{
    RI0=0;                //清接收中断标志
    *p=SBUF0;              //收到的字符送接收缓冲区
    p++;
    if(p>=0x40) ES0=0;     //关串口中断
}
```



3. 串行通信编程举例

• **例5** 试编写一个UART0带奇偶校验的发送程序。

• 设SYSCLK=11.0592MHz, 波特率=9600, UART0工作于方式1, 发送字符的**ASCII码最高位作校验位**, 用T2作波特率发生器, T2的时间常数计算如下:

$$\diamond \therefore \text{波特率} = \frac{\text{SYSCLK}}{32 \times (65536 - [\text{RCAP2H}: \text{RCAP2L}])}$$

$$\diamond \therefore \text{RCAP2H}: \text{RCAP2L} = 65536 - \frac{\text{SYSCLK}}{32 \times \text{波特率}}$$

$$= 65500 = 0\text{xffdc}$$



```
#include <c8051f020.h>
#include <string.h>
char s[]="C8051F020 Serial Communication";
char bdata c;    //可位寻址的内部数据存储区
sbit c7=c^7;    //因为后应需要针对C的最高位操作。
void main(void)
{
    char a,b=0;
    XBR0=0x04;    //配置交叉开关, 使能UART0
    XBR2=0x40;    //使能交叉开关
    P0MOUT=0x0f;    ///TX0为推挽输出(错), 总是漏极开路
    T2CON=0x14;    //T2作发送波特率发生器,TCLK0=1,TR2=1
    SCON0=0x40;    //SM20=SM00=0,SM10=1,REN0=0;
    RCAP2H=0xff;
    RCAP2L=0xdc;
    a=strlen(s);
```



```
for(;b<a;b++)  
{  
    c=s[b];  
    ACC=c;           //将字符送到累加器ACC中进行校验，形成校验位P  
    c7=P;           //最高位作校验位，P即PSW.0；ASCII码最高位恒为0  
    SBUF0=c;  
    while(!TI0);  
    TI0=0;  
}  
}
```

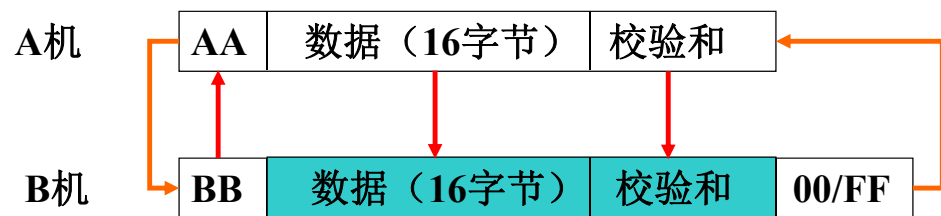
方式2、3中也可以使用TB8、RB8作校验位，请自己练习！



例6 A机和B机进行点对点通信

- A机开始发送时，先发送**AA联络信号**，
- B机收到后回答一个**BB信号**，表示同意接收。
- A机收到BB信号后开始发送数据，每发送一个字节便求一次**累加校验和**。
- 假定数据块长度为16字节，数据缓冲区首地址为BUF，数据发送完后马上发送校验和。
- B机接收数据并将其存入BUF为首地址的缓冲区，每收到一字节也计算一次校验和，当收齐数据块后，再接收A机发来的校验和，并将其与自己计算出来的**校验和进行比较**。若二者相等，则说明接收正确，B机**回答00H**；否则说明接收不正确，B机**回答FFH**，请求重发。
- A机收到00H的回答后，结束发送。否则将数据再重发一次。
- 双方约定的波特率为1200，SYSCLK=11.0592MHz，用定时器T1作波特率发生器，取T1M=0、SMOD=0。





•因为点对点通信的双方基本等同，只是人为规定一个为发送方，另一个为接收方。可以编制含有初始化函数、发送函数、接受函数的程序，在主程序中根据程序的发送和接受设置选择变量TR，根据TR的值选择要调用的函数。A机为TR = 0，B机为TR=1。




```
#include <c8051f020.h>
#define uchar unsigned char
#define TR 1          //发送、接收控制变量，0发送、1接收
uchar idata buf[16];  //发送、接收缓冲区
uchar pf;             //校验和
void init(void)       //初始化函数
{
    XBR0=0x04;         //配置交叉开关，使能UART0
    XBR2=0x40;         //使能交叉开关
    P0MOUT|=0x01;      ///TX0为推挽输出（错），总是漏极开路
    TMOD=0x20;         //定时器T1方式2，作波特率发生器
    TH1=0xe8;
    TL1=0xe8;
    PCON=0x00;
    TR1=1;
    SCON=0x50;         //UART0方式1、REN0=1
}
```



```
void send(uchar idata *d)           //发送函数
{
    uchar i;
    do {
        SBUF0=0xAA;                 //发送联络信号
        while(!TI0);
        TI0=0;
        while(!RI0);                //等待完成对方的应答信号0xBB
        RI0=0;
    }while((SBUF0^0xBB)!=0); //直到对方同意接收; “^” 为逐位异或
    do{
        pf=0;
        for(i=0;i<16;i++)
        {
            SBUF0=d[i];             //发送一字节
            pf+=d[i];               //计算校验和
            while(!TI0);
            TI0=0;
        }
    }
```



```
SBUF0=pf;           //发送校验和
while(!TI0);
TI0=0;
while(!RI0);         //等待对方回应
RI0=0;
}while(SBUF0!=0);    //直到对方回应00，否则重发
}
```



```
void receive(uchar idata *d)      //
```

接收函数

```
{  uchar i;
```

```
  do {
```

```
      while(!RI0);  RI0=0;
```

```
  }while((SBUF0^0xAA)!=0);
```

//接收发送请求

```
SBUF0=0XBB;
```

//回送应答

```
while(!TI0);  TI0=0;
```



While(1)

```
{
    pf=0;
    for(i=0;i<16;i++)
    {
        while(!RI0);           //接收并保存一个字符、计算校验和
        RI0=0;
        d[i]=SBUF0;
        pf+=d[i];
    }
    while(!RI0);               //接收校验和
    RI0=0;
    If((SBUF0^pf)==0)
    {
        SBUF0=0x00;           //回送接收正确应答
        while(!TI0);
        TI0=0;Break;
    }else
    { SBUF0=0xff;               //回送接收出错应答，请求重发
      while(!TI0);  TI0=0;
    }
}
```



A 机定义TR=0; B机定义TR=1

```
void main(void)    //主程序
```

```
{
```

```
    init();
```

```
    if(!TR)
```

```
        send(buf);    //TR=0,发送
```

```
    else
```

```
        receive(buf); //TR=1,接收
```

```
}
```

